

LAPORAN TUGAS BESAR
IF2224 TEORI BAHASA FORMAL DAN OTOMATA

MILESTONE 1



Disusun oleh:

Gabriella Botimada Lubis - 13524006

Stefani Angeline Oroh - 13524064

Reva Natania Sitohang - 13524098

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
BANDUNG

2026

DAFTAR ISI

DAFTAR ISI.....	3
DAFTAR TABEL.....	4
DAFTAR GAMBAR.....	5
BAB I	
DASAR TEORI.....	6
1.1 Bahasa Pemrograman dan Proses Kompilasi.....	6
1.2 Lexical Analysis.....	6
1.2.1 Fungsi dan Peran Lexer dalam Proses Kompilasi.....	6
1.2.2 Token.....	7
1.2.3 Cara Kerja Lexer.....	8
1.3 Deterministic Finite Automata (DFA).....	8
BAB II	
PERANCANGAN DAN IMPLEMENTASI.....	9
2.1 Diagram DFA.....	10
2.1.1 Daftar Token Bahasa Pemrograman Airon.....	11
2.2 Implementasi Program.....	18
2.2.1 Perancangan Program.....	18
2.3 Implementasi Kode.....	22
2.3.1 Implementasi Token.....	22
2.3.2 Implementasi DFA.....	22
2.3.3 Implementasi Lexer.....	25
BAB III	
PENGUJIAN.....	27
3.1 Test Case 1.....	27
3.2 Test Case 2.....	28
3.3 Test Case 3.....	30
3.4 Test Case 4.....	34
3.5 Test Case 5.....	37
3.6 Test Case 6.....	42
3.7 Test Case 7.....	43
BAB IV	
KESIMPULAN DAN SARAN.....	46
4.1 Kesimpulan.....	46
4.2 Saran.....	46
LAMPIRAN.....	48

DAFTAR TABEL

Tabel 2.1.1 Token Bahasa Pemrograman Airon.....	15
Tabel 3.1 Input/Output Test Case 1.....	27
Tabel 3.2 Input/Output Test Case 2.....	28
Tabel 3.3 Input/Output Test Case 3.....	32
Tabel 3.4 Input/Output Test Case 4.....	34
Tabel 3.5 Input/Output Test Case 5.....	38
Tabel 3.6 Input/Output Test Case 6.....	39
Tabel 3.7 Input/Output Test Case 7.....	41

DAFTAR GAMBAR

Gambar 2.1 (1) Diagram DFA.....	10
Gambar 2.1 (2) Lanjutan Diagram DFA.....	10
Gambar 2.3.1 Implementasi Token.....	21
Gambar 2.3.2 (1) State DFA pada Kode.....	22
Gambar 2.3.2 (2) Implementasi Diagram Transisi DFA.....	24
Gambar 2.3.3 Implementasi Lexer.....	24

BAB I

DASAR TEORI

1.1 Bahasa Pemrograman dan Proses Kompilasi

Bahasa pemrograman adalah sistem notasi formal yang digunakan untuk mengekspresikan algoritma dan instruksi yang dapat dieksekusi oleh komputer. Karena CPU hanya memahami bahasa mesin (binary 0 dan 1), diperlukan suatu mekanisme untuk menerjemahkan kode sumber yang ditulis manusia menjadi kode mesin. Terdapat dua pendekatan utama dalam proses ini, yaitu *compiler* dan *interpreter*. *Compiler* menerjemahkan seluruh kode sumber menjadi kode mesin sebelum program dieksekusi, menghasilkan file objek yang siap dijalankan. Sementara itu, *interpreter* memproses dan mengeksekusi kode sumber secara langsung baris per baris tanpa menghasilkan file objek terlebih dahulu. Meskipun cara kerjanya berbeda, kedua pendekatan ini pada dasarnya melewati tahapan analisis yang serupa, yaitu *Lexical Analysis*, *Syntax Analysis*, *Semantic Analysis*, dan *Intermediate Code Generation*.

1.2 Lexical Analysis

Lexical analysis adalah tahapan pertama dan paling mendasar dalam proses kompilasi atau interpretasi suatu bahasa pemrograman. Pada tahap ini, kode sumber yang pada dasarnya hanyalah serangkaian karakter mentah dibaca dan diubah menjadi rangkaian satuan-satuan bermakna yang disebut token. Proses ini dilakukan oleh komponen yang disebut lexer atau tokenizer.

1.2.1 Fungsi dan Peran Lexer dalam Proses Kompilasi

Lexer memiliki beberapa fungsi utama dalam proses kompilasi, antara lain:

- Memisahkan karakter-karakter mentah dari kode sumber menjadi unit-unit bermakna (*token*) yang lebih mudah diproses oleh tahap selanjutnya (*parsing*).
- Membuang bagian yang tidak diperlukan dalam proses kompilasi, seperti spasi, tab, baris baru, dan komentar, sehingga tahap berikutnya tidak perlu menanganinya.
- Melakukan validasi awal terhadap karakter-karakter yang dikenali. Apabila terdapat karakter atau urutan karakter yang tidak sesuai dengan pola token yang valid, lexer dapat mendeteksi dan melaporkan kesalahan leksikal (*lexical error*) sejak dini.

- Mengubah representasi string menjadi nilai semantik yang lebih efisien, misalnya mengubah string angka '42' menjadi nilai integer 42, serta menormalisasi *identifier* yang bersifat *case-insensitive*.

Dalam *pipeline* kompilasi/interpretasi, lexer berperan sebagai gerbang pertama yang menyederhanakan kompleksitas kode sumber sebelum diserahkan ke tahap parser. Dengan adanya lexer, parser tidak perlu lagi berurusan dengan karakter individual, melainkan langsung bekerja dengan aliran token yang sudah terstruktur dan bermakna.

1.2.2 Token

Token merupakan unit terkecil yang memiliki makna dalam sebuah program. Setiap token terdiri dari dua komponen utama, yaitu:

- Jenis token (*token type*), yang merupakan kategori atau klasifikasi token, misalnya keyword, identifier, operator, atau konstanta numerik.
- Nilai token (*token value/lexeme*), yang merupakan nilai aktual yang dibaca dari kode sumber, misalnya angka '42', nama variabel 'myVar', atau operator '+'.

Beberapa jenis token umum dalam bahasa pemrograman meliputi:

- *Keyword/Reserved Words* adalah kata-kata yang memiliki makna khusus dalam bahasa pemrograman dan tidak dapat digunakan sebagai identifier, seperti begin, end, if, while, for, dan lain-lain.
- *Identifier* adalah nama yang diberikan oleh programmer untuk variabel, konstanta, fungsi, prosedur, dan tipe data.
- Konstanta adalah nilai tetap yang ditulis langsung dalam kode, mencakup konstanta integer (intcon), konstanta bilangan riil (realcon), konstanta karakter (charcon), dan string (strcon).
- Operator adalah simbol yang merepresentasikan operasi aritmatika, logika, atau perbandingan, seperti +, -, *, /, :=, dan sebagainya.
- Tanda baca (*Punctuation/Delimiter*) adalah simbol-simbol struktural seperti tanda kurung, koma, titik koma, dan titik dua.
- Komentar (*Comment*) adalah teks yang ditujukan untuk dokumentasi dan diabaikan oleh *compiler/interpreter*.

1.2.3 Cara Kerja Lexer

Lexer bekerja dengan membaca kode sumber karakter demi karakter dari kiri ke kanan. Proses ini menggunakan pola-pola tertentu, yang biasanya diimplementasikan sebagai DFA untuk mengenali berbagai jenis token. Secara umum, alur kerja lexer adalah sebagai berikut:

1. Lexer membaca satu karakter dari kode sumber.
2. Berdasarkan karakter tersebut, lexer menentukan jenis token yang sedang diproses.
3. Lexer terus membaca karakter berikutnya selama karakter tersebut masih merupakan bagian dari token yang sama.
4. Ketika lexer menemukan karakter yang tidak lagi menjadi bagian dari token saat ini (misalnya spasi atau operator setelah identifier), token yang telah terkumpul dihasilkan (emit) dan lexer kembali ke kondisi awal untuk memproses token berikutnya.
5. Proses ini berulang hingga seluruh kode sumber selesai diproses.

1.3 Deterministic Finite Automata (DFA)

Deterministic Finite Automata (DFA) adalah model komputasi matematis yang digunakan untuk mengenali pola dalam string masukan. Secara formal, DFA didefinisikan sebagai sebuah 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$ di mana:

- Q : Himpunan berhingga dari state (kondisi)
- Σ : Alfabet masukan (himpunan simbol yang dapat dibaca)
- $\delta: Q \times \Sigma \rightarrow Q$, fungsi transisi yang deterministik (untuk setiap pasangan state dan simbol, ada tepat satu state berikutnya)
- $q_0 \in Q$: State awal
- $F \subseteq Q$: Himpunan state penerima (*accepting states*).

Deterministik berarti bahwa untuk setiap state dan setiap simbol masukan, terdapat tepat satu transisi yang mungkin terjadi. Hal ini berbeda dengan NFA (*Nondeterministic Finite Automata*) yang memperbolehkan beberapa transisi atau transisi epsilon untuk satu kondisi yang sama.

DFA memiliki kaitan yang sangat erat dengan implementasi lexer. Setiap pola *token* dalam bahasa pemrograman dapat direpresentasikan sebagai *regular expression*, dan setiap *regular expression* dapat dikonversi menjadi DFA yang ekuivalen. Oleh karena itu, lexer pada dasarnya adalah implementasi dari sekumpulan DFA yang digabungkan untuk mengenali semua jenis *token* yang valid. Dalam implementasi lexer berbasis DFA:

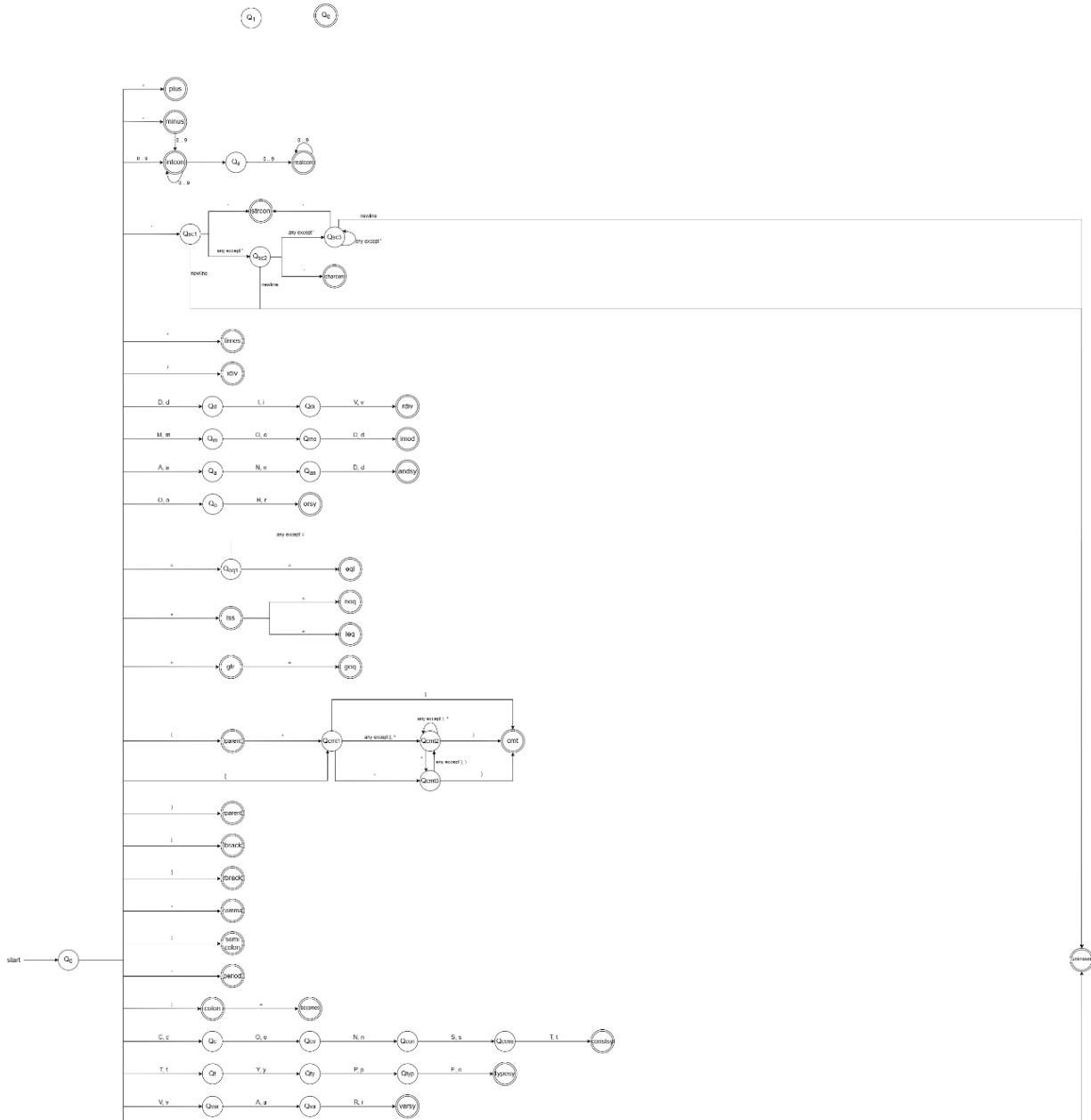
- Setiap state dalam DFA merepresentasikan kondisi atau konteks pembacaan saat ini.
- State awal (q_0) adalah kondisi awal sebelum ada karakter yang dibaca.
- Fungsi transisi menentukan state berikutnya berdasarkan karakter yang dibaca saat ini.
- State penerima (*accepting state*) merepresentasikan kondisi di mana sebuah token telah berhasil dikenali.
- Ketika DFA mencapai *accepting state* dan karakter berikutnya tidak dapat melanjutkan token, token tersebut dihasilkan dan DFA di *reset* ke state awal.

Implementasi lexer dengan DFA meningkatkan efisiensi, di mana setiap karakter hanya perlu diproses sekali ($O(n)$ di mana n adalah panjang input), dan determinisme DFA memastikan bahwa tidak ada ambiguitas dalam proses tokenisasi.

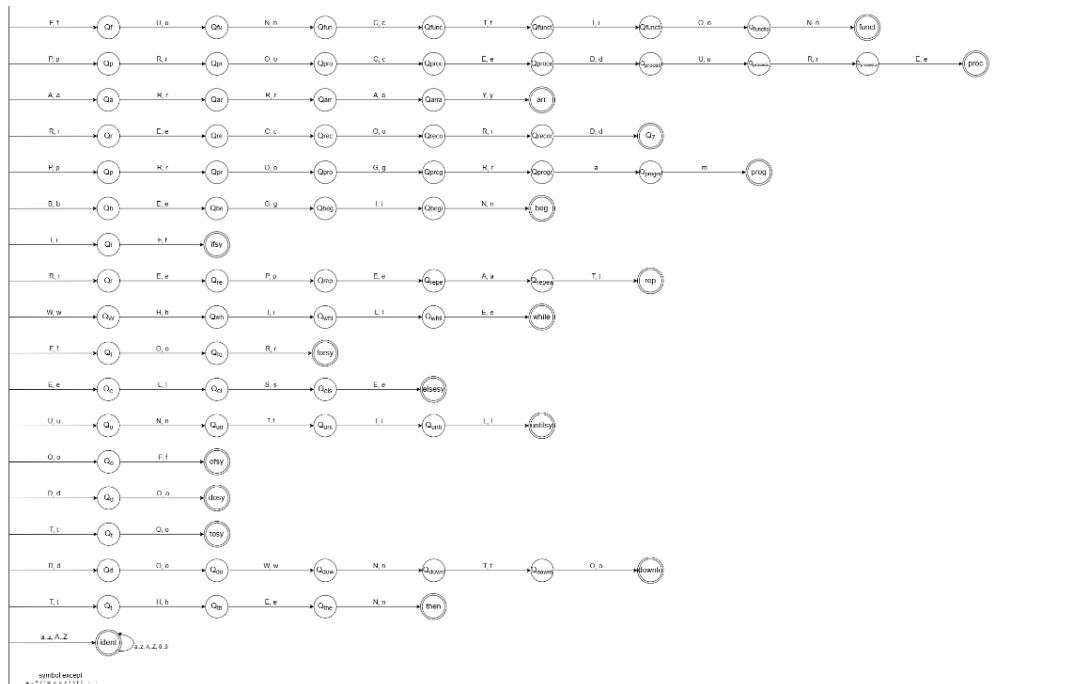
PERANCANGAN DAN IMPLEMENTASI

2.1 Diagram DFA

Berikut merupakan gambar dari diagram DFA yang telah kelompok kami buat:



Gambar 2.1 (1) Diagram DFA



Gambar 2.1 (2) Lanjutan Diagram DFA

2.1.1 Daftar Token Bahasa Pemrograman Airon

Bahasa pemrograman Arion mendefinisikan 52 jenis token yang harus dikenali oleh lexer. Token-token ini terbagi ke dalam beberapa kategori, yaitu konstanta/literal, operator aritmatika, operator logika, operator relasional, delimiter/tanda baca, kata kunci (*keyword*), *identifer*, dan komentar. Tabel berikut merangkum seluruh token beserta definisi dan contoh penggunaannya.

No	Token	Definisi & Keterangan	Contoh Pemakaian
1	intcon	Konstanta integer	1, 3, 48
2	realcon	Konstanta bilangan riil	3.14, 26.7

3	charcon	Konstanta karakter singular, diapit dengan tanda petik tunggal (')	'j', 'k', 't'
4	strcon	Sekuens karakter (string), diapit dengan tanda petik tunggal (')	'IRK', 'TBFO'
5	notsy	Operator logika NOT (negasi)	NOT
6	plus	Operator aritmatika pertambahan	+
7	minus	Operator aritmatika pengurangan	-
8	times	Operator aritmatika perkalian	*
9	idiv	Operator aritmatika pembagian Integer	div
10	rdiv	Operator aritmatika pembagian Riil	/
11	imod	Operator aritmatika modulo	MOD
12	andsy	Operator logika AND	AND
13	orsy	Operator logika OR	OR

14	eql	Operator sama dengan (equal)	=
15	neq	Operator tidak sama dengan (not equal)	$\neq$
16	gtr	Operator lebih besar dari (greater than)	>
17	geq	Operator lebih besar atau sama dengan (greater than or equal)	$\geq$
18	lss	Operator lebih kecil dari (less than)	<
19	leq	Operator lebih kecil atau sama dengan (less than or equal)	$\leq$
20	lparent	Tanda kurung kiri	(
21	rparent	Tanda kurung kanan	)
22	lbrack	Kurung siku kiri	[
23	rbrack	Kurung siku kanan	]
24	comma	Koma	,

25	semicolon	Titik koma	;
26	period	Titik	.
27	colon	Titik dua	:
28	becomes	Operator assignment	:=
29	constsy	Kata kunci deklarasi konstanta	const
30	typesy	Kata kunci deklarasi tipe data	type
31	varsy	Kata kunci deklarasi variabel	var
32	functionsy	Kata kunci deklarasi fungsi	function
33	proceduresy	Kata kunci deklarasi prosedur	procedure
34	arraysy	Kata kunci deklarasi array	array
35	recordsy	Kata kunci deklarasi record	record
36	programsy	Kata kunci deklarasi program	program

37	ident	Nama identifier (variabel, konstanta, tipe, prosedur, fungsi). Diawali huruf, diikuti huruf/angka, case-insensitive.	x, PI, MyInt, h4g
38	beginsy	Kata kunci begin (awal blok)	begin
39	ifsy	Kata kunci percabangan if	if
40	casesy	Kata kunci percabangan case	case
41	repeatsy	Kata kunci perulangan repeat	repeat
42	whilesy	Kata kunci perulangan while	while
43	forsy	Kata kunci perulangan for	for
44	endsy	Kata kunci akhir blok	end
45	elsesy	Kata kunci percabangan else	else
46	untilsy	Kata kunci akhir repeat-until	until
47	ofsy	Kata kunci of	of

48	dosy	Kata kunci do	do
49	tosy	Kata kunci to (for loop)	to
50	downtosy	Kata kunci downto (for loop menurun)	downto
51	thensy	Kata kunci then (setelah if)	then
52	comment	Komentar program, dimulai dengan { atau (* dan diakhiri dengan } atau *)	{ Ini komentar }, (* Ini komentar *)

Tabel 2.1.1 Token Bahasa Pemrograman Airon

2.1.2 Penjelasan Diagram Transisi DFA

Diagram transisi DFA yang dirancang untuk lexer bahasa Arion mencakup seluruh jenis token di atas. Berikut adalah penjelasan bagian-bagian utama DFA tersebut berdasarkan diagram yang telah dibuat.

a. State Awal (Q0)

State Q0 adalah state awal DFA, yaitu titik awal sebelum karakter apapun dibaca. Dari Q0, DFA dapat bertransisi ke berbagai state lain tergantung pada karakter pertama yang dibaca. Beberapa transisi langsung dari Q0 menghasilkan token single-character (seperti +, -, *, /, (,), [,], ,, ;) yang langsung masuk ke *accepting state* dan mengembalikan token yang sesuai.

b. Pengenalan Konstanta Numerik (intcon dan realcon)

Ketika Q0 membaca digit angka (0-9), DFA berpindah ke state intcon yang merupakan state untuk pengenalan intcon (konstanta integer). Selama karakter berikutnya masih berupa digit, DFA tetap di intcon. Jika kemudian dibaca karakter titik (.), DFA berpindah ke state Q_{ir} yang menandakan awal pembacaan bagian desimal. Jika setelah titik terdapat digit lagi, DFA berpindah ke state realcon yang

merupakan accepting state untuk realcon (konstanta bilangan riil). Selama digit terus dibaca di realcon, DFA tetap di realcon.

c. Pengenalan Konstanta Karakter dan String (charcon dan strcon)

Ketika Q0 membaca karakter tanda petik tunggal ('), DFA berpindah ke state Q_{sc1} . Dari Q_{sc1} , DFA membaca karakter apapun kecuali tanda petik tunggal untuk mengenali isi literal. Jika langsung diikuti tanda petik tunggal penutup, token yang dihasilkan adalah charcon (konstanta karakter tunggal). Jika ada lebih dari satu karakter sebelum tanda petik penutup, token yang dihasilkan adalah strcon (string). Pembedaan antara charcon dan strcon dilakukan saat DFA mencapai *accepting state* berdasarkan panjang konten yang telah dikumpulkan.

d. Pengenalan Operator Relasional

Operator relasional memerlukan penanganan khusus karena beberapa operator terdiri dari dua karakter. Misalnya, karakter '<' dapat menjadi token lss (jika hanya '<') atau neq (jika '<>') atau leq (jika '<='). DFA menangani ini dengan state intermediate, setelah membaca '<', DFA masuk ke state yang menunggu karakter berikutnya. Jika karakter berikutnya adalah '>', dihasilkan token neq; jika '=', dihasilkan leq; jika karakter lain, dihasilkan lss dan karakter tersebut dikembalikan untuk diproses ulang. Hal serupa berlaku untuk '>' (menghasilkan gtr atau geq), ':' (menghasilkan colon atau becomes), dan '=' (menghasilkan eql).

e. Pengenalan Identifier dan Keyword

Ketika Q0 membaca huruf (a-z atau A-Z), DFA berpindah ke state yang menangani *identifier* dan *keyword*. DFA terus membaca karakter alphanumeric (huruf dan angka) hingga bertemu karakter yang bukan alphanumeric. Setelah seluruh kata terkumpul, kata tersebut dicocokkan dengan daftar keyword bahasa Arion (case-insensitive) menggunakan *keyword lookup table*. Pada diagram DFA yang dibuat, pengenalan *keyword* dilakukan melalui jalur transisi karakter per karakter yang membentuk kata kunci tersebut. Jika cocok dengan salah satu *keyword*, token keyword yang sesuai dihasilkan, sebaliknya jika tidak cocok, token ident dihasilkan beserta nilainya.

f. Pengenalan Komentar

Bahasa Arion mendukung dua gaya komentar: komentar yang dibungkus oleh { dan }, serta komentar yang dibungkus oleh (* dan *). Ketika DFA membaca '{' dari Q0, DFA masuk ke state komentar dan terus membaca karakter apapun kecuali '}'.

Ketika '}' dibaca, komentar selesai dan DFA kembali ke Q0 tanpa menghasilkan token (atau menghasilkan token comment yang diabaikan). Untuk komentar gaya (*...*), setelah '(' dibaca dan diikuti '*', DFA masuk ke state komentar yang menunggu urutan '*' diikuti ')' sebagai penutup.

g. State Error/Unknown

DFA juga memiliki state unknown yang dicapai ketika terdapat karakter atau urutan karakter yang tidak valid dalam bahasa Arion. Ketika mencapai state error, lexer melaporkan kesalahan leksikal (*lexical error*) beserta informasi masukan apa yang tidak dikenali, sehingga pengguna dapat dengan mudah mengidentifikasi dan memperbaiki kode sumber mereka.

2.2 Implementasi Program

2.2.1 Perancangan Program

Program diimplementasikan menggunakan bahasa C++ dengan penggunaan beberapa Standard Library seperti <string> untuk manipulasi string (lexeme), <map> untuk penyimpanan keyword, <queue> untuk buffer token tambahan, <fstream> untuk membaca file input, dan <cctype> untuk klasifikasi karakter (huruf, digit, dll).

Struktur Program:

OTH-TUBES-IF2224-2026/

```
├── src/
│   ├── lexer/
│   │   ├── lexer.cpp
│   │   ├── lexer.hpp
│   │   ├── token.cpp
│   │   └── token.hpp
│   ├── test/
│   ├── main.cpp
│   └── main.exe
└── test/
```

```
| └─ milestone-1/
| └─ doc/
| └─ Laporan-1-OTH.pdf
└─ Makefile
```

Struktur Program adalah sebagai berikut dan dibagi menjadi beberapa komponen utama, yaitu :

1. Token.hpp

Berisi defisiisi enum TokenType (daftar seluruh jenis token), struct Token untuk representasi 1 token, dan fungsi tokenTypeToString() untuk konversi ke output string.

2. Lexer.hpp

Berisi deklarasi class Lexer, termasuk atribut untuk menyimpan state dan posisi deklarasi fungsi utama lexer, dan deklarasi helper function.

3. Lexer.cpp

Inti dari sistem lexical analysis

4. main.cpp

Beberapa fungsi atau kelas utama dalam program ini adalah sebagai berikut:

- Class lexer

Class Lexer merupakan komponen utama dalam melakukan proses lexical analysis. Class ini mengelola seluruh proses pembacaan input, pengaturan state DFA, pembentukan lexeme, dan menghasilkan token. Di dalamnya terdapat atribut penting seperti posisi pembacaan (line, col), state saat ini (state), dan juga mekanisme penyimpanan karakter sementara (pendingChar) untuk mendukung pembacaan satu karakter per langkah.

- Fungsi nextToken()

Fungsi ini adalah inti dari proses lexer yang menghasilkan satu token setiap kali dipanggil. Fungsi ini menjalankan DFA dengan membaca karakter satu per satu, menentukan transisi state, dan membangun lexeme menggunakan buffer. Ketika state akhir tercapai, fungsi ini akan mengembalikan token yang sesuai dengan jenis lexeme yang telah terbentuk.

- Struktur Token dan TokenType

Struktur Token digunakan untuk menyimpan hasil akhir dari proses lexical analysis, yang terdiri dari tipe token, nilai lexeme, dan posisi token dalam source code. TokenType mendefinisikan seluruh jenis token yang dikenali oleh lexer, seperti keyword, operator, konstanta, dan identifier.

- Fungsi readChar() dan setPending()

Fungsi readChar() membaca satu karakter dari file input secara berurutan. Fungsi ini memperbarui posisi baris dan kolom. Fungsi setPending() menyimpan satu karakter agar dapat dibaca kembali pada proses berikutnya. Mekanisme ini memungkinkan lexer tetap bekerja sesuai prinsip DFA tanpa menggunakan lookahead.

- Fungsi classifyIdent()

Fungsi ini digunakan untuk membedakan antara identifier dan keyword. Setelah DFA selesai membaca lexeme yang terdiri dari huruf dan angka, lexeme tersebut akan dibandingkan dengan tabel keyword. Jika ditemukan dalam tabel, maka token diklasifikasikan sebagai keyword, jika tidak, maka diklasifikasikan sebagai IDENT.

Alur kerja program untuk Milestone 1, Lexical Analysis ini adalah sebagai berikut:

1. Program dimulai dengan membuka file input yang berisi source code. Lexer diinisialisasi dengan state awal (S0) dan posisi awal pembacaan. Selain itu, tabel keyword juga disiapkan untuk digunakan dalam proses klasifikasi identifier.
2. Program membaca input menggunakan fungsi readChar() yang mengambil satu karakter setiap kali dipanggil. Pendekatan ini memastikan proses berjalan sesuai dengan prinsip DFA, yaitu membaca input secara sequential tanpa langsung melihat karakter berikutnya.
3. Setiap karakter yang dibaca menentukan perpindahan state DFA. Jika karakter pertama berupa huruf, maka lexer akan berpindah ke state identifier, jika berupa digit, maka ke state bilangan. Transisi ini diimplementasikan menggunakan struktur switch-case, yang merepresentasikan diagram DFA dalam bentuk kode.

4. Selama berada dalam state tertentu, lexer akan menambahkan karakter ke dalam buffer untuk membentuk lexeme. Proses ini berlangsung hingga lexer mencapai kondisi di mana token telah lengkap, yaitu saat karakter berikutnya tidak sesuai dengan pola token.
5. Setelah lexeme selesai dibentuk, lexer menentukan jenis token berdasarkan state akhir. Untuk identifier, dilakukan pengecekan tambahan menggunakan tabel keyword. Untuk token lain seperti angka, string, dan operator, jenis token langsung ditentukan dari state DFA.
6. Jika lexer menemukan pola yang tidak valid, seperti unclosed string, unclosed comment, atau karakter yang tidak dikenali, maka token akan diklasifikasikan sebagai UNKNOWN.
7. Token yang telah dikenali akan dikembalikan oleh fungsi nextToken() dan kemudian dicetak dalam format output yang sesuai, yaitu token atau token(value). Proses ini diulang hingga lexer menghasilkan EOF_TOKEN, tanda bahwa seluruh input telah selesai diproses.

2.3 Implementasi Kode

2.3.1 Implementasi Token



```
1  #ifndef TOKEN_H
2  #define TOKEN_H
3
4  #include <string>
5  using namespace std;
6
7  enum TokenType {
8      INTCON, REALCON, CHARCON, STRING,
9      NOTSY, PLUS, MINUS, TIMES, IDIV, RDIV, IMOD,
10     ANDSY, ORSY, EQL, NEQ, GTR, GEQ, LSS, LEQ,
11     LPARENT, RPARENT, LBRACK, RBRACK, COMMA, SEMICOLON, PERIOD, COLON,
12     BECOMES, CONSTSY, TYPESY, VARSY, FUNCTIONSY, PROCEDURESY,
13     ARRAYSY, RECORDSY, PROGRAMSY, IDENT, BEGINSY,
14     IFSY, CASESY, REPEATSY, WHILESY, FORSY, ENDSY, ELSESY, UNTILSY,
15     OFSY, DOSY, TOSY, DOWNTOSY, THENSY, COMMENT,
16     UNKNOWN, EOF_TOKEN
17 };
18
19 struct Token{
20     TokenType type;
21     string value;
22     int line;
23     int col;
24 };
25
```

Gambar 2.3.1 Implementasi Token

Token.h berfungsi sebagai definisi dasar dari struktur token yang akan dipakai di program lexical analysis. Enumerasi TokenType digunakan untuk merepresentasikan jenis token agar tidak disimpan sebagai teks bebas. Struktur Token dipakai untuk menyimpan informasi dari satu token yang berhasil dikenali oleh lexer. Struktur ini menyimpan tipe token (Type), lexeme atau string asli token (value), nomor baris token mulai ditemukan (line), dan nomor kolom awal token pada baris (col). Fungsi tokenTypeToString(TokenType type) digunakan untuk mengubah nilai enum TokenType menjadi bentuk yang dapat dibaca dalam representasi string yang akan dicetak di output.

2.3.2 Implementasi DFA

1. State DFA pada Kode

```

1  enum DFAState {
2      S0,
3
4      S_IDENT,
5
6      S_MINUS,
7      S_INT,
8      S_INT_DOT,      // sudah baca ddd.
9      S_REAL,
10     S_INT_NEG,
11     S_INT_NEG_DOT,   // sudah baca -ddd.
12     S_REAL_NEG,      // membaca digit desimal negatif
13
14
15     S_STR_OPEN,      // baru baca petik pertama
16     S_STR_BODY,      // di dalam isi string
17     S_STR_QUOTE,     // baru baca '\\' di dalam string , bisa escape or penutup
18
19     S_COLON,
20     S_LT,
21     S_GT,
22     S_EQL,
23
24     S_LPAREN,        // baru baca '('
25     S_CMT_BRACE,     // di dalam komentar '{'
26     S_CMT_BRACE_STAR, // di dalam '{', baru baca '*' , menunggu ')'
27
28     S_CMT_PAREN,     // di dalam komentar '('
29     S_CMT_PAREN_STAR, // di dalam '(', baru baca '*' , menunggu ')'
30
31     S_DEAD           // d

```

Gambar 2.3.2 (1) State DFA pada Kode

2. Implementasi Diagram Transisi DFA

```

1  while (true) {
2      char c = readChar();
3
4      if (state == S0 && !isWhitespace(c) && c != (char)EOF) {
5          tokLine = currentLine;
6          tokCol = currentCol;
7      }
8
9      switch (state) {
10
11     case S0: {
12         if (c == (char)EOF) {
13             return {EOF_TOKEN, "", tokLine, tokCol};
14         }
15
16         if (isWhitespace(c)) {
17             break;
18         }
19
20         if (isDigit(c)) {
21             buffer += c;
22             state = S_INT;
23             break;
24         }
25
26         if (isLetter(c)) {
27             buffer += c;
28             state = S_IDENT;
29             break;
30         }
31
32         if (c == '-') {
33             buffer += c;
34             state = S_MINUS;
35             break;
36         }
37
38         if (c == '+') {
39             return {PLUS, "", tokLine, tokCol};
40         }
41
42         if (c == '\\') {
43             state = S_STR_OPEN;
44             break;
45         }
46
47         if (c == '{') {
48             state = S_CMT_BRACE;
49             break;
50         }
51
52         if (c == '(') {
53             state = S_LPAREN;
54             break;
55         }
56
57         if (c == ':') {
58             state = S_COLON;
59             break;
60         }
61
62         if (c == '<') {
63             state = S_LT;
64             break;
65         }
66
67         if (c == '>') {
68             state = S_GT;
69             break;
70         }
71
72         if (c == '=') {
73             state = S_EQ;
74             break;
75         }
76

```

Gambar 2.3.2 (2) Implementasi Diagram Transisi DFA

Konsep DFA diimplementasikan ke dalam bentuk state machine pada kode. Setiap token dibaca dengan prinsip yang sama seperti DFA, yaitu:

1. lexer berada pada suatu state awal,
2. membaca satu karakter dari input,
3. menentukan transisi ke state berikutnya berdasarkan karakter tersebut,
4. berhenti ketika mencapai state final atau state error.

Secara implementasi, state DFA direpresentasikan dengan variabel state, sedangkan transisinya direpresentasikan melalui switch-case di dalam fungsi nextToken(). Dengan demikian, diagram transisi DFA yang dibuat pada tahap perancangan dapat langsung dipetakan ke alur program. Implementasi DFA dilakukan melalui enum DFA State di lexer.hpp yang mendefinisikan state-state seperti S0 (start), S_IDENT, S_INT, hingga S_DEAD untuk error, serta logika transisi di fungsi nextToken() pada lexer.cpp. DFA memproses input karakter demi karakter dari file sumber, mengubah state berdasarkan karakter saat ini, dan menghasilkan token saat mencapai kondisi accepting (misalnya, mengembalikan INTCON setelah membaca digit).

2.3.3 Implementasi Lexer



```
1  Lexer::Lexer(const std::string& filename)
2      : currentLine(1), currentCol(0),
3        hasPending(false), pendingChar('\0'),
4        state(S0)
5  {
6      inputFile.open(filename);
7      if (!inputFile.is_open()) {
8          throw std::runtime_error("Cannot open input file: " + filename);
9      }
10 }
```

Gambar 2.3.3 Implementasi Lexer

Lexer diimplementasikan sebagai komponen utama yang bertugas membaca file input .txt, menjalankan DFA di atas input tersebut, lalu menghasilkan token satu per satu. Secara struktur, lexer tidak langsung memproses seluruh file menjadi daftar token sekaligus, melainkan menyediakan fungsi `nextToken()` yang dipanggil berulang untuk mengambil token berikutnya.

Constructor digunakan untuk membuka file input, inisialisasi posisi awal pembacaan, inisialisasi state awal DFA, dan menyiapkan `pendingChar`. Variabel `currentLine` dan `currentCol` digunakan untuk mencatat posisi token pada source code, `hasPending` dan `pendingChar` digunakan untuk satu karakter simpanan, dan `state(S0)` menunjukkan bahwa lexer mulai dari state awal.

BAB III

PENGUJIAN

3.1 Test Case 1

Masukan (input)
<pre>program Hello; var a, b: integer; begin a := 5; b := a + 10; writeln('Result = ', b); end.</pre>
Keluaran (output)
<pre>programsy ident(Hello) semicolon varsy ident(a) comma ident(b) colon ident(integer) semicolon beginsy ident(a) becomes intcon(5) semicolon ident(b) becomes ident(a)</pre>

```

plus
intcon(10)
semicolon
ident(writeln)
lparent
string(Result = )
comma
ident(b)
rparent
semicolon
endsy
period

```

Tabel 3.1 Input/Output Test Case 1

3.2 Test Case 2

Masukan (input)

```

program DataTypes;
const
  PI := 3.14;
  MAX := 100;
  GRADE := 'A';
  MSG := 'Hello World';
  MSG2 := 'Hello ''World''';
  EMPTYSTR := '';
var
  x: real;
  n: integer;
  c: char;
  c2: char;
  s: string;
begin
  x := +26.7;
  n := -42;
  c := 'c';
  c2 := '';

```

<pre> s := 'IRK'; end. </pre>
Keluaran (output)
<pre> programsy ident (Hello) semicolon varsy ident (a) comma ident (b) colon ident (integer) semicolon beginsy ident (a) becomes intcon (5) semicolon ident (b) becomes ident (a) plus intcon (10) semicolon ident (writeln) lparent string (Result =) comma ident (b) rparent semicolon endsy period </pre>

Tabel 3.2 Input/Output Test Case 2

3.3 Test Case 3

Masukan (input)
<pre>program Operators; var a, b, hasil: integer; flag: boolean; begin a := 20; b := 3; hasil := a + b; hasil := a - b; hasil := a * b; hasil := a div b; hasil := a mod b; hasil := a / b; flag := (a == b); flag := (a <> b); flag := (a > b); flag := (a >= b); flag := (a < b); flag := (a <= b); flag := (flag and true); flag := (flag or false); flag := not flag; end.</pre>
Keluaran (output)
<pre>programsy ident(Operators) semicolon varsy ident(a) comma ident(b) comma ident(hasil)</pre>

```
colon
ident(integer)
semicolon
ident(flag)
colon
ident(boolean)
semicolon
beginsy
ident(a)
becomes
intcon(20)
semicolon
ident(b)
becomes
intcon(3)
semicolon
ident(hasil)
becomes
ident(a)
plus
ident(b)
semicolon
ident(hasil)
becomes
ident(a)
minus
ident(b)
semicolon
ident(hasil)
becomes
ident(a)
times
ident(b)
semicolon
ident(hasil)
becomes
ident(a)
idiv
```

```
ident (b)
semicolon
ident (hasil)
becomes
ident (a)
imod
ident (b)
semicolon
ident (hasil)
becomes
ident (a)
rdiv
ident (b)
semicolon
ident (flag)
becomes
lparent
ident (a)
eq1
ident (b)
rparent
semicolon
ident (flag)
becomes
lparent
ident (a)
neq
ident (b)
rparent
semicolon
ident (flag)
becomes
lparent
ident (a)
gtr
ident (b)
rparent
semicolon
```

```
ident(flag)
becomes
lparent
ident(a)
geq
ident(b)
rparent
semicolon
ident(flag)
becomes
lparent
ident(a)
lss
ident(b)
rparent
semicolon
ident(flag)
becomes
lparent
ident(a)
leq
ident(b)
rparent
semicolon
ident(flag)
becomes
lparent
ident(flag)
andsy
ident(true)
rparent
semicolon
ident(flag)
becomes
lparent
ident(flag)
orsy
ident(false)
```



```
rparent  
semicolon  
ident(flag)  
becomes  
notsy  
ident(flag)  
semicolon  
endsy  
period
```

Tabel 3.3 Input/Output Test Case 3

3.4 Test Case 4

Masukan (input)

```
program ControlFlow;  
var  
    i, n: integer;  
begin  
    { contoh if-else }  
    if n > 0 then  
        writeln('positif')  
    else  
        writeln('non-positif');  
  
    { contoh while }  
    i := 1;  
    while i <= 10 do  
        begin  
            i := i + 1;  
        end;  
  
    { contoh for }  
    for i := 1 to 5 do  
        writeln(i);  
  
    { contoh repeat-until }
```

```

repeat
    i := i - 1;
until i == 0;

{ contoh case }
case i of
    1: writeln('satu');
    2: writeln('dua');
end;
end.

```

Keluaran (output)

```

programsy
ident(ControlFlow)
semicolon
varsy
ident(i)
comma
ident(n)
colon
ident(integer)
semicolon
beginsy
comment(contoh if-else)
ifsy
ident(n)
gtr
intcon(0)
thensy
ident(writeln)
lparent
string(positif)
rparent
elsesy
ident(writeln)
lparent
string(non-positif)

```

```
rparent
semicolon
comment(contoh while)
ident(i)
becomes
intcon(1)
semicolon
whilesy
ident(i)
leq
intcon(10)
dosy
beginsy
ident(i)
becomes
ident(i)
plus
intcon(1)
semicolon
endsy
semicolon
comment(contoh for)
forsy
ident(i)
becomes
intcon(1)
tosy
intcon(5)
dosy
ident(writeln)
lparent
ident(i)
rparent
semicolon
comment(contoh repeat-until)
repeatsy
ident(i)
becomes
```

```

ident(i)
minus
intcon(1)
semicolon
untilsy
ident(i)
eq1
intcon(0)
semicolon
comment(contoh case)
casesy
ident(i)
ofsy
intcon(1)
colon
ident(writeln)
lparent
string(satu)
rparent
semicolon
intcon(2)
colon
ident(writeln)
lparent
string(dua)
rparent
semicolon
endsy
semicolon
endsy
period

```

Tabel 3.4 Input/Output Test Case 4

3.5 Test Case 5

Masukan (input)

```

program Structures;
type
  Point = record
    x: integer;
    y: integer;
  end;
  Arr = array [1..10] of integer;

function tambah(a, b: integer): integer;
begin
  tambah := a + b;
end;

procedure cetakHasil(hasil: integer);
begin
  writeln(hasil);
end;

var
  p: Point;
  data: Arr;
  res: integer;
begin
  p.x := 3;
  p.y := 4;
  data[1] := 99;
  res := tambah(p.x, p.y);
  cetakHasil(res);
end.

```

Keluaran (output)

```

programsy
ident(Structures)
semicolon
sypesy
ident(Point)
unknown(=)

```

```
recordsy
ident(x)
colon
ident(integer)
semicolon
ident(y)
colon
ident(integer)
semicolon
endsy
semicolon
ident(Arr)
unknown(=)
arraysy
lbrack
intcon(1)
period
period
intcon(10)
rbrack
ofsy
ident(integer)
semicolon
functionsy
ident(tambah)
lparent
ident(a)
comma
ident(b)
colon
ident(integer)
rparent
colon
ident(integer)
semicolon
beginsy
ident(tambah)
becomes
```

```
ident(a)
plus
ident(b)
semicolon
endsy
semicolon
proceduresy
ident(cetakHasil)
lparent
ident(hasil)
colon
ident(integer)
rparent
semicolon
beginsy
ident(writeln)
lparent
ident(hasil)
rparent
semicolon
endsy
semicolon
varsy
ident(p)
colon
ident(Point)
semicolon
ident(data)
colon
ident(Arr)
semicolon
ident(res)
colon
ident(integer)
semicolon
beginsy
ident(p)
period
```

```
ident(x)
becomes
intcon(3)
semicolon
ident(p)
period
ident(y)
becomes
intcon(4)
semicolon
ident(data)
lbrack
intcon(1)
rbrack
becomes
intcon(99)
semicolon
ident(res)
becomes
ident(tambah)
lparent
ident(p)
period
ident(x)
comma
ident(p)
period
ident(y)
rparent
semicolon
ident(cetakHasil)
lparent
ident(res)
rparent
semicolon
endsy
period
```


Tabel 3.5 Input/Output Test Case 5

3.6 Test Case 6

Masukan (input)
<pre> { komentar kurung kurawal } (* komentar kurung bintang *) { komentar kurung buka dan tutup berbeda *) program KomentarTest; var NamaVariabel: INTEGER; HASIL: Real; begin (* komentar multi baris (* ini masih masuk *) ini tidak *) NamaVariabel := 7; HASIL := NamaVariabel / 2; { komentar di tengah kode } WRITELN(HASIL); end.</pre>
Keluaran (output)
<pre> comment(komentar kurung kurawal) comment(komentar kurung bintang) comment(komentar kurung buka dan tutup berbeda) programsy ident(KomentarTest) semicolon varsy ident(NamaVariabel) colon ident(INTEGER) semicolon ident(HASIL) colon ident(Real)</pre>

```

semicolon
beginsy
comment(komentar
      multi
      baris (* ini masih masuk)
ident(ini)
ident(tidak)
times
rparent
ident>NamaVariabel)
becomes
intcon(7)
semicolon
ident(HASIL)
becomes
ident>NamaVariabel)
rdiv
intcon(2)
semicolon
comment(komentar di tengah kode)
ident(WRITELN)
lparent
ident(HASIL)
rparent
semicolon
endsy
period

```

Tabel 3.6 Input/Output Test Case 6

3.7 Test Case 7

Masukan (input)
<pre> program ErrorTest; var x: integer; begin </pre>

```

x := 5;

{ karakter tidak dikenal }
x := x @ 2;
x := x # 1;

{ string tidak ditutup }
writeln('ini string tidak ditutup);

{ komentar tidak ditutup
x := 10;

{ penggunaan = tunggal (bukan == atau :=) }
x = 5;

{ angka diikuti huruf langsung - ident terpisah }
x := 3abc;
end.

```

Keluaran (output)

```

programsy
ident(ErrorTest)
semicolon
varsy
ident(x)
colon
ident(integer)
semicolon
beginsy
ident(x)
becomes
intcon(5)
semicolon
comment(karakter tidak dikenal)
ident(x)
becomes
ident(x)

```

```

unknown(@)
intcon(2)
semicolon
ident(x)
becomes
ident(x)
unknown(#)
intcon(1)
semicolon
comment(string tidak ditutup)
ident(writeln)
lparent
unknown('ini string tidak ditutup);)
comment(komentar tidak ditutup
    x := 10;

    { penggunaan = tunggal (bukan == atau :=))
ident(x)
unknown(=)
intcon(5)
semicolon
comment(angka diikuti huruf langsung - ident terpisah)
ident(x)
becomes
intcon(3)
ident(abc)
semicolon
endsy
period

```

Tabel 3.7 Input/Output Test Case 7

BAB IV

KESIMPULAN DAN SARAN

4.1 Kesimpulan

Pada Milestone 1 ini, kelompok kami berhasil merancang dan mengimplementasikan sebuah lexer untuk bahasa pemrograman Arion menggunakan pendekatan *Deterministic Finite Automata* (DFA). Lexer yang dibangun mampu membaca kode sumber karakter demi karakter dan menghasilkan aliran token yang terstruktur sebagai fondasi untuk tahap kompilasi berikutnya

Bahasa pemrograman Arion mendefinisikan 52 jenis token yang mencakup konstanta (intcon, realcon, charcon, strcon), operator aritmatika, operator logika, operator relasional, delimiter, 23 kata kunci (*keyword*), *identifier*, dan komentar. Seluruh token tersebut berhasil dikenali dan diklasifikasikan dengan benar oleh lexer yang diimplementasikan. Implementasi DFA dilakukan melalui enumerasi DFASState di lexer.hpp dan logika transisi di fungsi nextToken() pada lexer.cpp. Setiap state merepresentasikan satu kondisi pembacaan, mulai dari S0 (state awal) hingga S_DEAD (*state error*), sehingga diagram transisi DFA yang dirancang dapat langsung dipetakan ke struktur kode.

Pengujian dilakukan melalui 7 test case yang mencakup berbagai skenario, seperti pengenalan tipe data dasar, operator, struktur kontrol, fungsi dan prosedur, komentar dengan berbagai format, serta penanganan karakter dan pola yang tidak valid. Hasil pengujian menunjukkan bahwa lexer mampu menangani kasus-kasus tersebut sesuai dengan spesifikasi yang ditetapkan. Secara keseluruhan, Milestone 1 ini berhasil menghasilkan komponen lexer yang fungsional, efisien ($O(n)$ terhadap panjang input), dan siap dikembangkan lebih lanjut sebagai bagian dari sistem kompilator Arion pada milestone-milestone berikutnya.

4.2 Saran

Berdasarkan dari pengalaman mengerjakan Milestone 1, terdapat beberapa saran yang dapat menjadi pertimbangan untuk pengembangan di milestone-milestone selanjutnya, antara lain:

1. Pesan kesalahan leksikal yang dihasilkan dapat diperkaya dengan informasi baris dan kolom secara konsisten di seluruh jenis error, sehingga pengguna lebih mudah mengidentifikasi dan memperbaiki kode sumber mereka.
2. Pengujian ke depan sebaiknya mencakup kasus-kasus ekstrem seperti *identifier* dengan panjang sangat panjang, angka dengan digit yang banyak, serta string kosong, untuk memastikan ketangguhan lexer dalam segala kondisi.
3. Struktur data token yang dihasilkan lexer sebaiknya dirancang untuk mudah dikonsumsi oleh parser pada Milestone 2.
4. Setiap fungsi dan bagian penting dari kode sebaiknya diberi komentar yang jelas dan informatif, menjelaskan tujuan, parameter, dan nilai kembalian sehingga anggota tim dapat memahami dan mengembangkan kode pada milestone-milestone berikutnya dengan baik.
5. Biasakan menggunakan *branching strategy* yang baik pada repository GitHub, misalnya dengan membuat branch terpisah untuk setiap fitur atau milestone, sehingga riwayat pengembangan lebih rapi dan mudah dilacak.

LAMPIRAN

Link Repository : <https://github.com/gabriellaalubis/OTH-Tubes-IF2224-2026>

Link Workspace Diagram DFA :

<https://drive.google.com/file/d/1vuRwWDh7q-4-7Osm5fuxl5PFzbJxzP16/view>

Pembagian tugas :

Nama	NIM	Pembagian	Persentase
Gabriella Botimada Lubis	13524006	● Perancangan DFA ● Implementasi Kode ● Pengujian ● Penyusunan Laporan	33.3%
Stefani Angeline Oroh	13524064	● Perancangan DFA ● Implementasi Kode ● Pengujian ● Penyusunan Laporan	33.3%
Reva Natania Sitohang	13524098	● Perancangan DFA ● Implementasi Kode ● Pengujian ● Penyusunan Laporan	33.3%